Sesión 12

Sistemas de Recomendación:
Filtrado colaborativo, factorización matricial y métricas

Carlos César Sánchez Coronel

MACHINE LEARNING

Contenido

1. Introducción
2. Tipos de Sistemas de Recomendación
3. Factorización Matricial: Funk SVD
4. Métricas de Evaluación
5. Problemas y Desafíos
6. Caso Integrado: MovieLens
7. Implementación en Python
8. Aplicaciones en la Industria
9. Conexión y Resumen

Contenido

1. Introducción
2. Tipos de Sistemas de Recomendación
3. Factorización Matricial: Funk SVD
4. Métricas de Evaluación
5. Problemas y Desafíos
6. Caso Integrado: MovieLens
7. Implementación en Python
8. Aplicaciones en la Industria
9. Conexión y Resumen

La revolución de la personalización

- Sistemas de recomendación: núcleo de plataformas como Netflix, Amazon, Spotify, TikTok.
- Objetivo: predecir qué ítems serán más relevantes para un usuario.
- **Datos explícitos**: valoraciones, likes, comentarios.
- **Datos implícitos**: clics, tiempo de visualización, compras.
- Impacto: ¿35 % de ventas en Amazon, reducción de cancelaciones en Netflix.

Contenido

1. Introducción
2. Tipos de Sistemas de Recomendación
3. Factorización Matricial: Funk SVD
4. Métricas de Evaluación
5. Problemas y Desafíos
6. Caso Integrado: MovieLens
7. Implementación en Python
8. Aplicaciones en la Industria
9. Conexión y Resumen

Basados en popularidad

Idea

Recomendar los mismos ítems a todos los usuarios según métricas globales (más vistos, más comprados).

- **Ventaja**: simple, útil para cold start del sistema.
- **Desventaja**: no personaliza, poca diversidad.

Basados en contenido (Content-based)

Idea

Recomendar ítems similares a los que el usuario ha valorado positivamente, usando características de los ítems.

- Cada ítem i se representa con un vector de características $\mathbf{x}_i$ (género, director, palabras clave).
- Perfil del usuario $\mathbf{p}_u = \frac{1}{|I_u^+|} \sum_{i \in I_u^+} \mathbf{x}_i$.
- Relevancia: similitud de coseno $\dfrac{\mathbf{p}_u \cdot \mathbf{x}_j}{\|\mathbf{p}_u\| \|\mathbf{x}_j\|}$.

Filtrado colaborativo (Collaborative Filtering)

Principio

Usuarios con gustos similares en el pasado tendrán gustos similares en el futuro.

Basado en usuarios

- Encontrar usuarios similares al activo.
- Predecir valoraciones ponderando las de los vecinos.

Basado en ítems

- Calcular similitud entre ítems.
- Recomendar ítems similares a los que el usuario ya valoró.

Similitud de Pearson

$$\text{sim}(u, v) = \frac{\sum_{i \in I_{uv}} (r_{ui} - \bar{r}_u)(r_{vi} - \bar{r}_v)}{\sqrt{\sum_{i \in I_{uv}} (r_{ui} - \bar{r}_u)^2} \sqrt{\sum_{i \in I_{uv}} (r_{vi} - \bar{r}_v)^2}}$$

Predicción en filtrado colaborativo

Basado en usuarios

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in N(u)} \text{sim}(u, v)(r_{vi} - \bar{r}_v)}{\sum_{v \in N(u)} |\text{sim}(u, v)|}$$

- $N(u)$: vecinos más cercanos.
- Problemas de escalabilidad: calcular similitudes entre todos los pares es $O(n^2)$.
- Solución: indexación (LSH), reducción de dimensionalidad.

Sistemas híbridos

Combinación de enfoques

- Mitigan cold start.
- Mejoran precisión y diversidad.
- Ejemplo: Netflix combina contenido, colaborativo y factores contextuales.

Formas de hibridación: ponderada, conmutada, cascada, etc.

Contenido

1. Introducción
2. Tipos de Sistemas de Recomendación
3. Factorización Matricial: Funk SVD
4. Métricas de Evaluación
5. Problemas y Desafíos
6. Caso Integrado: MovieLens
7. Implementación en Python
8. Aplicaciones en la Industria
9. Conexión y Resumen

El problema de la matriz dispersa

- Matriz R de m usuarios $\times$ n ítems, con muchas celdas vacías.
- Objetivo: encontrar matrices P $(m \times k)$ y Q $(n \times k)$ tales que

$$R \approx PQ^T$$

- Cada fila $\mathbf{p}_u$ representa los gustos latentes del usuario.
- Cada fila $\mathbf{q}_i$ representa las características latentes del ítem.
- Predicción: $\hat{r}_{ui} = \mathbf{p}_u \cdot \mathbf{q}_i$.

Funk SVD (Simon Funk)

Idea

Optimizar solo sobre las valoraciones conocidas mediante gradiente descendente estocástico (SGD) con regularización.

Función de pérdida:

$$\mathcal{L} = \sum_{(u,i)\in\mathcal{K}} (r_{ui} - \mathbf{p}_u \cdot \mathbf{q}_i)^2 + \lambda(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2)$$

- $\mathcal{K}$: conjunto de valoraciones conocidas.
- λ: regularización (evita overfitting).

Actualización SGD

Para cada valoración conocida r_{ui}:

1. Error: $e_{ui} = r_{ui} - \mathbf{p}_u \cdot \mathbf{q}_i$
2. Actualizar $\mathbf{p}_u$: $\mathbf{p}_u \leftarrow \mathbf{p}_u + \eta(e_{ui}\mathbf{q}_i - \lambda\mathbf{p}_u)$
3. Actualizar $\mathbf{q}_i$: $\mathbf{q}_i \leftarrow \mathbf{q}_i + \eta(e_{ui}\mathbf{p}_u - \lambda\mathbf{q}_i)$

donde η es la tasa de aprendizaje.

Extensiones: sesgos (biases)

Modelo más realista:

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u \cdot \mathbf{q}_i$$

- μ: media global.
- b_u: sesgo del usuario (tendencia a valorar más alto/bajo).
- b_i: sesgo del ítem (ítem generalmente mejor/peor valorado).

Las actualizaciones incluyen también b_u y b_i con su propia regularización.

Contenido

1. Introducción
2. Tipos de Sistemas de Recomendación
3. Factorización Matricial: Funk SVD
4. Métricas de Evaluación
5. Problemas y Desafíos
6. Caso Integrado: MovieLens
7. Implementación en Python
8. Aplicaciones en la Industria
9. Conexión y Resumen

Métricas para predicción de rating

RMSE

$$\sqrt{\frac{1}{|\mathcal{K}|}\sum(r_{ui}-\hat{r}_{ui})^2}$$

Penaliza más los errores grandes.

MAE

$$\frac{1}{|\mathcal{K}|}\sum|r_{ui}-\hat{r}_{ui}|$$

Menos sensible a outliers.

- RMSE fue la métrica oficial del Premio Netflix.

Métricas para rankings: Precision@k y Recall@k

Definiciones

- **Precision@k**: $\frac{\text{número de ítems relevantes en top-}k}{k}$
- **Recall@k**: $\frac{\text{número de ítems relevantes en top-}k}{\text{total de ítems relevantes del usuario}}$

Ejemplo (Pregunta 9)

Usuario con 5 relevantes. Recomendación top-3: [A, B, C] con A y C relevantes.
Precision@3 $= 2/3 = 0.667$, Recall@3 $= 2/5 = 0.4$.

Compromiso: a mayor k, recall aumenta, precisión puede disminuir.

Mean Average Precision (MAP)

- Para un usuario, Average Precision (AP):

$$AP = \frac{\sum_{k=1}^{n} \text{Precision@k} \cdot rel(k)}{\text{número de relevantes}}$$

  donde $rel(k) = 1$ si el ítem en posición k es relevante.
- **MAP** es el promedio de AP sobre todos los usuarios.

Ejemplo (Pregunta 10)

MAPA = (0.6+0.8+0.5)/3 = 0.633; MAPB = 0.6. Se elige A.

NDCG (Normalized Discounted Cumulative Gain)

- DCG@k: $\sum_{i=1}^{k} \frac{2^{rel_i} - 1}{\log_2(i+1)}$
- NDCG normaliza dividiendo por el DCG ideal (IDCG) donde los relevantes están en las primeras posiciones.
- Penaliza que ítems relevantes aparezcan abajo en el ranking.

Cobertura (Coverage)

- **Cobertura de ítems**: porcentaje de ítems que el sistema puede recomendar.
- **Cobertura de usuarios**: porcentaje de usuarios que reciben recomendaciones personalizadas.
- Alta precisión sin cobertura ignora ítems nicho y perjudica a usuarios con gustos específicos.

Contenido

1. Introducción
2. Tipos de Sistemas de Recomendación
3. Factorización Matricial: Funk SVD
4. Métricas de Evaluación
5. Problemas y Desafíos
6. Caso Integrado: MovieLens
7. Implementación en Python
8. Aplicaciones en la Industria
9. Conexión y Resumen

Cold start

Nuevo usuario

- Sin historial.
- Soluciones: popularidad, pedir preferencias iniciales, datos demográficos.

Nuevo ítem

- Sin valoraciones.
- Soluciones: basado en contenido, metadatos.

Nuevo sistema

- Sin datos de interacciones.
- Soluciones: transfer learning, encuestas.

Escalabilidad y matriz dispersa

- Millones de usuarios e ítems $\rightarrow$ calcular similitudes entre todos es inviable.
- Técnicas:
  - Indexación aproximada (LSH, HNSW).
  - Factorización matricial con SGD (procesa solo valoraciones conocidas).
  - Precomputación offline.
- Regularización ayuda a generalizar con datos dispersos (grande si pocos datos).

Diversidad y serendipia

- **Diversidad**: variedad de ítems recomendados.
- **Serendipia**: recomendar ítems relevantes pero inesperados.
- Optimizar solo precisión puede crear "filtros burbuja".
- Soluciones: añadir exploración, combinar con contenido, usar MMR.

Contenido

1. Introducción
2. Tipos de Sistemas de Recomendación
3. Factorización Matricial: Funk SVD
4. Métricas de Evaluación
5. Problemas y Desafíos
6. Caso Integrado: MovieLens
7. Implementación en Python
8. Aplicaciones en la Industria
9. Conexión y Resumen

Dataset MovieLens 100k

- 100,000 valoraciones (1-5) de 943 usuarios sobre 1,682 películas.
- Cada usuario ha valorado al menos 20 películas.

Preprocesamiento

1. Crear matriz usuario-película.
2. División entrenamiento (80 %) / prueba (20 %) estratificada por usuario.
3. Opcional: simular cold start eliminando algunos usuarios/ítems.

Modelos y resultados típicos

Modelo	**RMSE**	**NDCG@10**
Media global + sesgos	0.98	0.72
Item-based CF	0.95	0.78
Funk SVD (k=20, =0.1)	0.91	0.83

- Funk SVD supera a los demás, mejor también en ranking.

Contenido

1. Introducción
2. Tipos de Sistemas de Recomendación
3. Factorización Matricial: Funk SVD
4. Métricas de Evaluación
5. Problemas y Desafíos
6. Caso Integrado: MovieLens
7. Implementación en Python
8. Aplicaciones en la Industria
9. Conexión y Resumen

Filtrado colaborativo con Surprise

```
from surprise import Dataset, Reader, SVD
from surprise.model_selection import train_test_split
from surprise import accuracy

reader = Reader(line_format='user item rating', sep='\t')
data = Dataset.load_from_file('u.data', reader=reader)
trainset, testset = train_test_split(data, test_size=0.2)

algo = SVD(n_factors=20, reg_all=0.1, lr_all=0.005, n_epochs=20)
algo.fit(trainset)
predictions = algo.test(testset)
rmse = accuracy.rmse(predictions)
mae = accuracy.mae(predictions)
```

Funk SVD desde cero con NumPy

```
import numpy as np

def funk_svd(R, k=20, lambda_reg=0.1, lr=0.005, epochs=20):
    m, n = R.shape
    P = np.random.normal(0, 0.1, (m, k))
    Q = np.random.normal(0, 0.1, (n, k))
    nonzeros = np.argwhere(~np.isnan(R))
    for epoch in range(epochs):
        np.random.shuffle(nonzeros)
        for u, i in nonzeros:
            rui = R[u, i]
            pred = np.dot(P[u], Q[i])
            eui = rui - pred
            P[u] += lr * (eui * Q[i] - lambda_reg * P[u])
            Q[i] += lr * (eui * P[u] - lambda_reg * Q[i])
    return P, Q
```

Cálculo de Precision@k

```
def precision_at_k(recommended_items, relevant_items, k):
    recommended_k = recommended_items[:k]
    hits = len(set(recommended_k) & set(relevant_items))
    return hits / k
```

Contenido

1. Introducción
2. Tipos de Sistemas de Recomendación
3. Factorización Matricial: Funk SVD
4. Métricas de Evaluación
5. Problemas y Desafíos
6. Caso Integrado: MovieLens
7. Implementación en Python
8. Aplicaciones en la Industria
9. Conexión y Resumen

Netflix, Amazon, Spotify, TikTok

- **Netflix**: sistema híbrido (colaborativo + contenido + contexto). Personaliza incluso carátulas.
- **Amazon**: filtrado colaborativo basado en ítems ("los que compraron esto también compraron...").
- **Spotify**: combina colaborativo, análisis de audio y playlists colaborativas (Descubrimiento Semanal).
- **TikTok**: deep learning con interacciones, tiempo de visualización y redes sociales.

Contenido

1. Introducción
2. Tipos de Sistemas de Recomendación
3. Factorización Matricial: Funk SVD
4. Métricas de Evaluación
5. Problemas y Desafíos
6. Caso Integrado: MovieLens
7. Implementación en Python
8. Aplicaciones en la Industria
9. Conexión y Resumen

Conexión con el resto del curso

- Factorización matricial (álgebra lineal, optimización SGD).
- Métricas Precision/Recall (extensión de clasificación a rankings).
- Cold start (relacionado con datos faltantes en feature engineering).
- Regularización (vista en regresión y boosting).

Lo que hemos aprendido

Conceptos clave

- Tipos: popularidad, contenido, colaborativo, híbrido.
- Factorización matricial: Funk SVD, sesgos, SGD, regularización.
- Métricas de rating: RMSE, MAE.
- Métricas de ranking: Precision@k, Recall@k, MAP, NDCG.
- Problemas: cold start, escalabilidad, dispersión, diversidad.
- Caso práctico: MovieLens, implementaciones en Python.
- Aplicaciones industriales: Netflix, Amazon, Spotify, TikTok.

¡Gracias!